



<https://www.linkedin.com/company/quantum-robotics-automation-research-group/?viewAsMember=true>

Artificial Intelligence

CSE-316

(Section: 65-D/H)

Instructor

Dr. M. Akhtaruzzaman

Associate Professor, CSE, DIU

Associate Professor and Head, RME, DIU

akhtaruzzaman.cse@diu.edu.bd;

akhter900@gmail.com

Theory Lecture-12

NATURAL LANGUAGE PROCESSING

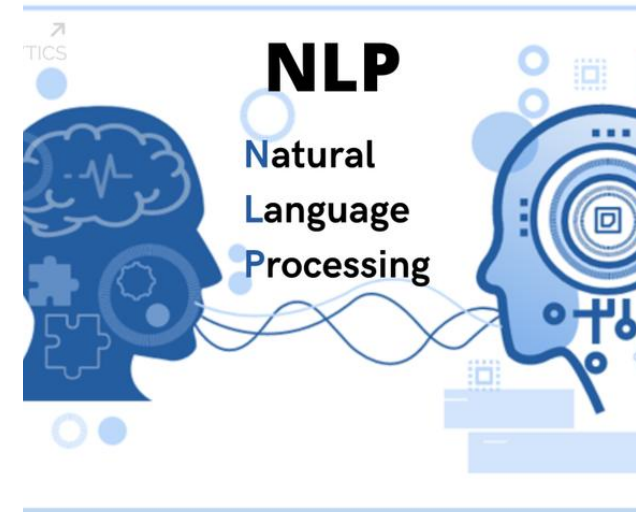
02. 04, 2026, DIU, Dhaka.

رب زدني علما وارزقني فهما.

My Lord! Advance me in Knowledge and True Understanding.

INTRODUCTION

Natural Language Processing is a subset of artificial intelligence that enables machines to comprehend and analyze human languages. Text or audio can be used to represent human languages.



GOALS

- Convert human language → machine-understandable format
- Extract meaning, patterns, and knowledge from text
- Enable machines to communicate with humans

EXAMPLES

- Chatbots (ChatGPT, Siri, Alexa)
- Google Translate
- Email spam detection
- Sentiment analysis
- Question answering systems

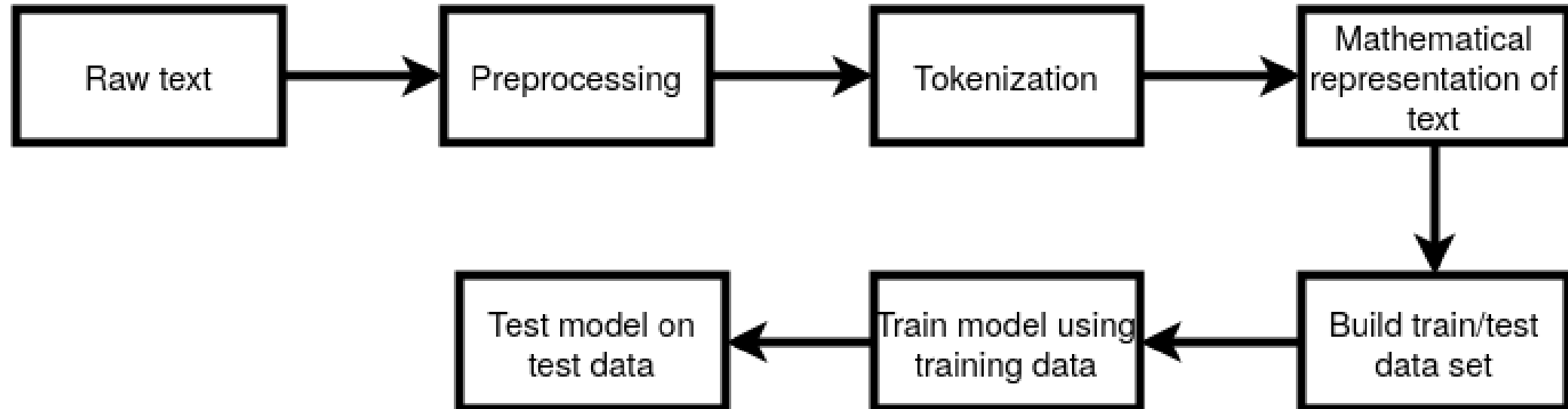
WHY NLP MATTERS

- NLP enables machines to interpret text and speech which are critical for search, chatbots, translators, summarizers, etc.. For example, chat assistants, spam filters, and voice interfaces rely on NLP.
- Data abundance: Over 80% of enterprise data is unstructured text (emails, social media, reports), so NLP unlocks business insights.
- Cross-domain impact: From healthcare (clinical notes) to finance (document analysis) to social media (sentiment).
- Trends: Deep learning and large pre-trained models (BERT, GPT) have dramatically boosted NLP capabilities

LINGUISTIC FUNDAMENTALS

- **Morphology:** Words are built from morphemes (smallest meaning units: roots, prefixes, suffixes).
- **Part-of-Speech (POS):** Each word falls into categories (noun, verb, etc.), which affects meaning. POS tagging assigns these categories, aiding syntactic parsing and disambiguation.
- **Syntax:** Grammar rules govern sentence structure (e.g. “The cat sat” vs. “Cat the sat”). Syntax includes parse trees and dependencies. Later used in advanced parsing models.
- **Semantics:** Concerned with meaning. Words can have multiple senses. Context and semantic networks are key for understanding synonyms/antonyms.
- **Impact on NLP:** Many feature extraction and model choices (e.g. lemmatizer vs. stemmer) derive from these linguistic concepts.

NLP PIPELINE



EXAMPLE SENTENCE PROCESSING

Input Sentence: "I love this movie so much!"

1.Raw Text: The original sentence provided by the user.
Result: "I love this movie so much!"

2. Preprocessing: Cleaning the text for easier processing.
Steps:

- Convert to lowercase
- Remove punctuation

Result: i love this movie so much

EXAMPLE SENTENCE PROCESSING

3. Tokenization: Split the sentence into individual tokens.

Result: ["i", "love", "this", "movie", "so", "much"]

Why it is important

- Makes text easier to process
- Helps later steps like tagging and vectorization
- Preserves structure for the model

EXAMPLE SENTENCE PROCESSING

4. Mathematical Representation: Convert words into numerical vectors.
Computers cannot work directly with words, so text must be converted into numbers.

Common representations

- Bag-of-Words
- TF-IDF
- Word embedding

Bag-of-Words (BoW)

The Bag of Words (BoW) model is the simplest form of text representation in numbers. Like the term itself, we can represent a sentence as a bag of words vector (a string of numbers).

Let's discuss three types of movie reviews:

Review 1: This movie is very scary and long

Review 2: This movie is not scary and is slow

Review 3: This movie is spooky and good

We will first build a vocabulary from all the unique words in the above three reviews. The vocabulary consists of these 11 words: 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'.

Bag-of-Words (BoW)

We can now take each of these words and mark their occurrence in the three movie reviews above with 1s and 0s. This will give us 3 vectors for 3 reviews:

	1 This	2 movie	3 is	4 very	5 scary	6 and	7 long	8 not	9 slow	10 spooky	11 good	Length of the review(in words)
Review 1	1	1	1	1	1	1	1	0	0	0	0	7
Review 2	1	1	2	0	0	1	1	0	1	0	0	8
Review 3	1	1	1	0	0	0	1	0	0	1	1	6

Bag-of-Words (BoW)

Vector-of-Review1: [1 1 1 1 1 1 1 0 0 0 0]

Vector-of-Review2: [1 1 2 0 0 1 1 0 1 0 0]

Vector-of-Review3: [1 1 1 0 0 0 1 0 0 1 1]

And that's the core idea behind a Bag of Words (BoW) model.

However, we start facing issues of Bag of Words when we come across new sentences:

- If the new sentences contain new words, then our vocabulary size would increase and thereby, the length of the vectors would increase too.
- Additionally, the vectors would also contain many 0s, thereby resulting in a sparse matrix (which is what we would like to avoid)
- We are retaining no information on the grammar of the sentences nor on the ordering of the words in the text.

TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) is a statistical method used in natural language processing and information retrieval to evaluate how important a word is to a document in relation to a larger collection of documents. TF-IDF combines two components:

1. **Term Frequency (TF):** Measures how often a word appears in a document. A higher frequency suggests greater importance. If a term appears frequently in a document, it is likely relevant to the document's content.

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

TF-IDF

2. Inverse Document Frequency (IDF): Reduces the weight of common words across multiple documents while increasing the weight of rare words. If a term appears in fewer documents, it is more likely to be meaningful and specific.

$$\text{IDF}(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

TF-IDF

Applications:

1. **Document Similarity and Clustering:** By converting documents into numerical vectors TF-IDF enables comparison and grouping of related texts. This is valuable for clustering news articles, research papers or customer support tickets into meaningful categories.
2. **Text Classification:** It helps in identify patterns in text for spam filtering, sentiment analysis and topic classification.
3. **Keyword Extraction:** It ranks words by importance making it possible to automatically highlight key terms, generate document tags or create concise summaries.
4. **Recommendation Systems:** Through comparison of textual descriptions TF-IDF supports suggesting related articles, videos or products enhancing user engagement.

Word Embedding

Word embeddings are a way of representing words as vectors in a multi-dimensional space, where the distance and direction between vectors reflect the similarity and relationships among the corresponding words.

The development of embeddings to represent text has played a crucial role in advancing natural language processing (NLP) and machine learning (ML) applications. Word embeddings have become integral to tasks such as text classification, sentiment analysis, machine translation and more.

Traditional methods of representing words in a way that machines can understand, such as one-hot encoding, represent each word as a sparse vector with a dimension equal to the size of the vocabulary. Here, only one element of the vector is "hot" (set to 1) to indicate the presence of that word.

Word Embeddings

Word embeddings, on the other hand, are dense vectors with continuous values that are trained using machine learning techniques, often based on neural networks. The idea is to learn representations that encode semantic meaning and relationships between words. Word embeddings are trained by exposing a model to a large amount of text data and adjusting the vector representations based on the context in which words appear.

One popular method for training word embeddings is Word2Vec, which uses a neural network to predict the surrounding words of a target word in a given context. Another widely used approach is GloVe (Global Vectors for Word Representation), which leverages global statistics to create embeddings.

Two approaches to word embeddings

1. **Frequency-based embeddings:** Frequency-based embeddings refer to word representations that are derived from the frequency of words in a corpus. These embeddings are based on the idea that the importance or significance of a word can be inferred from how frequently it occurs in the text.
2. **Prediction-based embeddings:** Prediction-based embeddings are word representations derived from models that are trained to predict certain aspects of a word's context or neighboring words. Unlike frequency-based embeddings that focus on word occurrence statistics, prediction-based embeddings capture semantic relationships and contextual information, providing richer representations of word meanings.

EXAMPLE SENTENCE PROCESSING

5. Build Train/Test Dataset: Many labeled sentences are used. Split dataset by 80/20%

Example training data:

Sentence	Label
I love this movie	Positive
This movie is terrible	Negative
Amazing acting	Positive
Waste of time	Negative

EXAMPLE SENTENCE PROCESSING

6. Train Model: Machine learning models learn patterns.

Common models

- Naive Bayes
- Logistic Regression
- Random Forest
- Deep Learning (LSTM / BERT)

The model learns that: love → positive sentiment

Learning goal

The model learns patterns such as:

- positive words → positive sentiment
- negative words → negative sentiment

This is where the mathematical representation becomes useful for prediction.

EXAMPLE SENTENCE PROCESSING

7. Test Model: The trained model predicts sentiment on unseen text.

Input: **"I love this movie so much!"**

Output: **Sentiment: Positive**

Confidence: **0.94**

Testing checks whether the model generalizes beyond the training examples. In classification tasks, performance depends on the chosen evaluation threshold and the dataset balance.

TRANSFORMERS AND BERT

Modern NLP is dominated by transformer-based models. The Transformer architecture is based on attention mechanisms and removes recurrence and convolutions. BERT is a bidirectional transformer pretrained with masked language modeling and next sentence prediction, and it can be fine-tuned for downstream NLP tasks.

Why transformers are powerful

- They capture long-range dependencies
- They process sequences more effectively
- They achieve strong results across many NLP tasks

TRANSFORMERS AND BERT

- **Static vs Contextual:** Unlike word2vec, contextual embeddings give each word occurrence a vector depending on its sentence. Allows polysemous words (e.g. “bank” as financial vs river) to have different vectors.
- **ELMo, BERT, GPT, etc.:** Early contextual models used BiLSTMs (ELMo), modern ones use Transformers. BERT (Devlin et al., 2018) is encoder-only, bidirectional Transformer using masked language modeling. GPT is decoder-only (autoregressive).
- **Transformer architecture:** Uses multi-head self-attention to encode all positions together. Key advantages are captures long-range dependencies and parallelizable. (Diagram: simplified transformer block or attention illustration.)
- **BERT specifics:** Pre-trained on huge corpora (English Wikipedia, BooksCorpus). Objective: predict masked tokens and next-sentence. Result: powerful language representations.
- **RoBERTa, DistilBERT, etc.:** Variants with tweaks (no NSP, larger data, distilled smaller models).
- **Use in NLP:** These models provide state-of-art results in many tasks. Often fine-tuned for tasks. Hugging Face Transformers library provides easy access.
- **Trade-offs:** Huge models → high accuracy but heavy compute and memory.

ATTENTION IN SIMPLE WORDS

Attention helps the model focus on the most relevant words in a sentence.

Example: “The movie was so good that I loved every minute of it.”
Attention helps the model understand that good, loved, and every minute carry strong sentiment signals.

Transformers use self-attention to examine the entire sequence at once, which helps them handle long context efficiently.

ATTENTION

- **Attention mechanism:** Allows a model to focus on different parts of input sequence when making each prediction. Improves long-range context handling. For example, in translation, attending to relevant source words.
- **Self-Attention:** Special case where input attends to itself. Each token's representation is a weighted sum of all tokens in sequence.
- **Transformers** (Vaswani et al. 2017): Replace recurrence with layers of multi-head self-attention + feed-forward networks. Process all tokens in parallel, capturing global context.
- **Multi-head:** Multiple attention “heads” learn different aspects of word relationships in parallel.
- **Advantages:** Handles long sequences efficiently, allows parallelization (fast training), achieves state-of-art in MT, summarization, etc.
- **Architecture:** Encoder (stack of self-attention + FFN layers) and decoder (with masked attention + encoder-decoder attention). BERT uses only encoder part.
- **Trade-offs:** Very large models requiring GPU/TPU; quadratic complexity in sequence length (attention cost).

TRANSFER LEARNING & FINE-TUNING

- **Transfer Learning in NLP:** Use a model pre-trained on large corpora and adapt it to a specific task. Saves time/data vs training from scratch. Common since BERT (2018) and GPT.
- **Fine-Tuning:** Add task-specific head (e.g. classification layer) and continue training on labeled data. The entire model (or most layers) is updated slightly on the new data. Highly effective for tasks like sentiment or NER.
- **Examples:** BERT fine-tuned on GLUE tasks (e.g. sentiment SST, NLI MNLI). Hugging Face makes this easy. Also feature-based: use embeddings as input to simpler model (less common now).
- **Benefits:** Leverages vast pretraining knowledge (grammar, facts). Often few-shot or one-shot learning possible. E.g., BERT reached 93% accuracy on SST with <10k examples.
- **Pre-trained Libraries:** Hugging Face Transformers, spaCy pretrained pipelines (NER/POS), TensorFlow Hub (ELMo).

EVALUATION METRICS

To judge NLP models, we use different metrics depending on the task.

For classification

- Accuracy
- Precision
- Recall
- F1 score

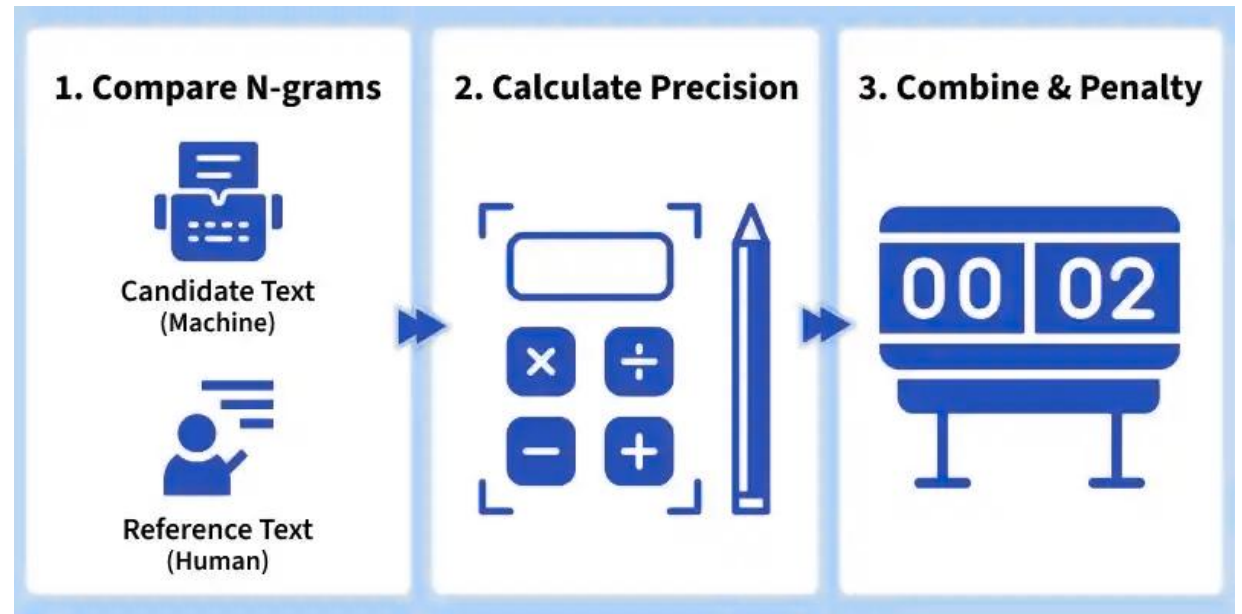
For generation tasks

- BLEU for translation
- ROUGE for summarization

BLEU AND ROUGE SCORE FOR NLP EVALUATION

BLEU Score

The BLEU (Bilingual Evaluation Understudy) score is a metric mainly used to evaluate machine translation systems. It measures how closely a machine-generated translation matches one or more human-written reference translations. The basic idea is that the more similar the candidate text is to the reference text, the better the translation quality.

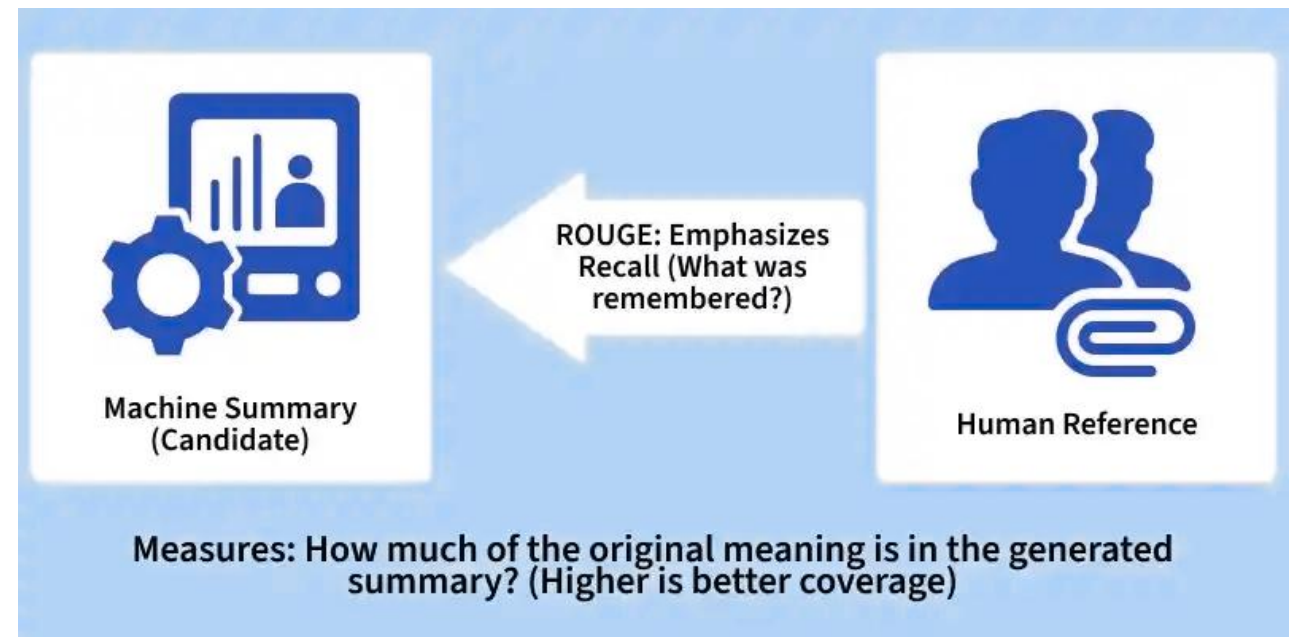


BLEU AND ROUGE SCORE FOR NLP EVALUATION

ROUGE Score

The ROUGE (Recall Oriented Understudy for Gisting Evaluation) score is mainly used to evaluate text summarization and other text generation tasks. It measures how much of the important information from the reference text is captured in the generated output. The final ROUGE score ranges from 0 to 1, where higher values indicate better content coverage and similarity to the reference text.

Unlike BLEU, which focuses more on precision, ROUGE emphasizes recall, meaning it checks how much relevant content is covered.



BLEU AND ROUGE SCORE FOR NLP EVALUATION

BLEU vs ROUGE

Aspect	BLEU	ROUGE
Main Focus	Precision (how much generated text matches reference)	Recall (how much reference content is covered)
Primary Use Case	Machine Translation	Text Summarization
Matching Method	n gram overlap with precision calculation	n gram, LCS and skip bigram overlap with recall emphasis
Length Handling	Uses brevity penalty for short outputs	No strict brevity penalty mechanism
Score Range	0 to 1 (higher is better)	0 to 1 (higher is better)

CHALLENGES IN NLP

NLP is powerful, but language is complex.

Challenges

- Ambiguity
- Sarcasm
- Misspellings
- Multilingual text
- Bias in data
- High computation cost for large models

Research has shown that language models and embeddings can carry and amplify bias, and privacy-sensitive text often requires anonymization or pseudonymization before use. Quantization is one way to reduce inference cost for transformer models.

CONCLUSION

NLP transforms raw language into structured representations, learns from data, and produces useful predictions.

Modern NLP combines linguistic processing, machine learning, and deep learning to understand language at scale.

REFERENCES

- spaCy Documentation
- NLTK Documentation and Book
- scikit-learn Text Feature Extraction and Train/Test Split
- Hugging Face BERT and Transformers Docs
- Transformer Paper (Attention Is All You Need)
- Google ML Crash Course on classification metrics
- ROUGE Python package
- <https://ieeexplore.ieee.org/abstract/document/10441408>
- <https://mijst.mist.ac.bd/mijst/index.php/mijst/article/view/440>
- <https://ieeexplore.ieee.org/abstract/document/10060927>
- <https://dl.acm.org/doi/full/10.1145/3723178.3723227>
- <https://ieeexplore.ieee.org/abstract/document/10941379>
- <https://ieeexplore.ieee.org/abstract/document/10939233>
- Apu, M. A., Shakil, U. A., & Akhtaruzzaman, M. (2022). Development of an Intelligent Chatbot for MIST Website. *CSE Technical Paper, The MIST Journal of Computer Science and Engineering*, 1(1), 33–36.

وَلَا تَلْبِسُوا الْحَقَّ بِالْبَاطِلِ وَتَكُنْهُوَ الْحَقَّ وَأَنْتُمْ تَعْلَمُونَ ﴿٤٢﴾

Do not mix truth with falsehood or hide the truth knowingly.

وَلَا تَلْبِسُوا الْحَقَّ بِالْبَاطِلِ وَتَكُنْهُوَ الْحَقَّ وَأَنْتُمْ تَعْلَمُونَ
(42) know [you] while you the Truth and conceal with [the] the Truth mix And (do) not

Thank You